TECHNICAL ENGINEERING REPORT

Kubernetes Projects: Mini to Production

Status: Completed & Verified
Runtime: Kind / Minikube v1.3x
Core Engine: Kubernetes v1.28+

Executive Abstract: This report documents the end-to-end design, implementation, container lifecycle resilience testing, persistent state migration, load autoscaling, packaging, and GitOps automation of a distributed multi-tier application. The workload evolves from basic primitive manifests to a production-calibrated Helm v3 chart continuously delivered via ArgoCD and automated with GitHub Actions.

1. Architecture & System Decomposition

The core application workload (Mini Project 1) implements a distributed, state-aware request and hit counter application configured to guarantee zero-downtime rolling upgrades and decouple ephemeral presentation from durable storage.

Key Structural Layers

- **Namespace Boundary (`mini-project-1`):** Provides logical isolation, scoped role-based access, and resource quota confinement.
- **Stateless Presentation Tier:** Deployed via `apps/v1 Deployment` using `ghcr.io/stefanprodan/podinfo:6.5.4` with 2 replicas, configured with rolling update parameters (`maxSurge: 1` , `maxUnavailable: 0`).
- **Decoupled Configuration & Credentials:** Non-sensitive settings (`REDIS_HOST` , `REDIS_PORT` , `APP_ENV`) are mapped via ConfigMaps, while credentials (`REDIS_PASSWORD`) are injected via Kubernetes Opaque Secrets.
- **Liveness & Readiness Health Probes:** HTTP `/healthz` and `/readyz` probes ensure deadlocked pods are restarted while non-ready instances are isolated from Service endpoints.

2. Workload Component Specifications

Component	Kubernetes Primitive	Hardware Bounds	Service / Network
Frontend Web	`Deployment` (2 replicas)	Req: 50m CPU, 32Mi Lim: 150m CPU, 64Mi	`ClusterIP` : Port 80 (`frontend-svc`)
Redis Storage	`StatefulSet` (1 replica)	Req: 50m CPU, 64Mi Lim: 200m CPU, 128Mi	`Headless` : Port 6379 (`redis-svc`)
L7 Routing	`Ingress` (NGINX)	N/A	Host: `myapp.local` → `frontend-svc:80`
Autoscaling	`HPA (autoscaling/v2)`	Min: 2 • Max: 6	Target: 50% CPU Utilization

3. Local Cluster Topology & Runtime Setup

Two containerized cluster environments were standardized to validate both multi-node control planes and developer productivity addons:

- **Kind (Kubernetes in Docker):** Deployed with 1 control-plane and 2 dedicated worker nodes via `kind-config.yaml`. Primary engine for high-velocity CI/CD test automation.
- **Minikube:** Configured using the Docker driver (2 CPUs, 4096 MiB Memory). Utilized for ingress controller testing and metric aggregation via native addons (`minikube addons enable ingress metrics-server`).

4. Resilience & Container Lifecycle Validation

Controlled failure injection scenarios were executed to observe declarative reconciliation mechanisms:

Failure Vector	Control Plane Reaction	Observed Result
Pod Deletion (`kubectl delete pod`)	ReplicaSet detects current replicas (1) < desired replicas (2).	Scheduler places a new replacement pod immediately; zero user-facing downtime.
SIGKILL PID 1 (`kill 1`)	Node Kubelet detects exit code 137 and executes `restartPolicy: Always`.	Pod retains name and IP; container is recreated in-place; `RESTARTS` counter increments.
Liveness Deadlock (`redis-cli shutdown`)	TCP port 6379 fails consecutive probes past `failureThreshold`.	Kubelet terminates deadlocked container. Demonstrated need for durable PVCs as ephemeral memory is cleared.

5. Stateful Workload Migration & Data Durability

To eliminate state loss inherent to basic Deployments, the caching tier was refactored into a `StatefulSet` coupled with a `volumeClaimTemplate` requesting 1Gi `ReadWriteOnce` persistent storage mounted to `/data` with Redis append-only logging (`--appendonly yes`). Durability was verified by writing test keys, deleting the pod instance (`redis-0`), and confirming complete key retention upon reattachment.

6. Helm v3 Packaging & Templating

Discrete YAML manifests were synthesized into a modular, production-ready Helm chart (`k8s-mini-project-1`) containing standardized helper templates (`_helpers.tpl`), conditional toggle flags for Ingress and HPA, resource bounds, and environment parameter overrides.

```
# Static chart syntax check and dry-run evaluation
helm lint ./k8s-mini-project-1
helm install mini-app ./k8s-mini-project-1 --dry-run --debug

# Declarative cluster deployment
helm install mini-app ./k8s-mini-project-1 -n mini-project-helm --create-namespace
```

7. Continuous Integration & GitOps Delivery

- **GitHub Actions CI (`.github/workflows/helm-ci.yaml`):** Automatically triggers on push and pull requests to validate chart syntax (`helm lint`), evaluate Go template macros, and spin up an ephemeral Kind cluster to test real installations before merging.
- **ArgoCD GitOps Automation:** Configured with `syncPolicy.automated.selfHeal=true` and `prune=true` . When manual configuration drift was induced by scaling pods to 0 via CLI, ArgoCD detected the out-of-sync state and automatically restored replicas to the Git declaration.

8. Operational Incident Log: Minikube DNS Resolution

Incident: Container images stalled under `ErrImagePull` and `ImagePullBackOff` .
Root Cause: Host network and VPN DNS resolvers leaked into Minikube's node runtime, yielding upstream DNS timeouts on port 53.
Resolution: Sideloaded images directly via `minikube image load` and updated the node's `/etc/resolv.conf` with public resolvers (`8.8.8.8` , `1.1.1.1`).